Clase 8 • Bootcamp AI Engineer

Ejecución Local: Cloud vs OSS + Trade-offs

Selección de backend • Modo privado/offline • Cuándo conviene cada opción

Ramsés Camas Nájera • Código Facilito

Agenda

/01 Modelos Open Source vs Open Weight vs Propietarios

/02 Parámetros: qué son y cómo medirlos

/03 Cálculo de memoria GPU requerida

/04 Cuantización: comprimiendo modelos

/05 Cloud vs OSS: matriz de decisión

/06 HuggingFace: el hub del ecosistema

/07 Herramientas: Ollama, vLLM, SGLang

/08 Extra: llama.cpp

/09 Código: inference/local_adapter.py

¿Qué es un modelo "Open Source"?

Técnica /01: Open Source vs Open Weight vs Propietario

3 categorías de modelos

Propietarios (GPT-4o, Claude, Gemini):
Acceso solo vía API. No conocemos arquitectura, datos de entrenamiento ni pesos.

Open Weight (Llama 3, Mistral, Qwen):
Publican los pesos para ejecutarlos, pero no datos de entrenamiento completos.

Open Source real (OLMo, Pythia):
Publican todo: pesos, datos, código de entrenamiento y documentación.

En la práctica...

Cuando decimos "modelos OSS" en la industria, nos referimos casi siempre a modelos open weight.

La distinción importa legalmente: cada modelo tiene una licencia que define si puedes usarlo comercialmente, modificarlo o redistribuirlo.

Siempre revisa la licencia antes de integrar un modelo en producción.

⭐ Revisa SIEMPRE la licencia: Apache 2.0, Llama License, Gemma License tienen restricciones distintas.

¿Por qué importa el OSS para AI Engineers?

Ventajas estratégicas de los modelos abiertos

Privacidad y compliance

Los datos nunca salen de tu infraestructura. Para salud, gobierno o finanzas, esto no es opcional.

Control total

Fine-tuning con tus datos, cuantizar al nivel que necesites, sin depender de actualizaciones de terceros.

Costos predecibles

Pagas infraestructura (GPU), no tokens. A alto volumen el costo por token se desploma.

Riesgo

Eres responsable del mantenimiento, actualizaciones de seguridad, escalamiento y monitoreo.

¿Qué son los parámetros de un modelo?

Parámetros — La unidad fundamental de un LLM

Un parámetro es un valor numérico que el modelo aprendió durante el entrenamiento. Cada peso en cada capa de atención, cada bias, cada embedding: todo es un parámetro. Cuando decimos "Llama 3 70B" significa que el modelo tiene ~70 mil millones de valores numéricos.

Más parámetros = mayor capacidad, pero también más memoria, cómputo y latencia.

Rango	Tamaño	Uso típico
Pequeños	1B – 3B	Tareas específicas, edge, móvil
Medianos	7B – 14B	Workhorses de producción
Grandes	32B – 70B	Razonamiento complejo
Masivos	100B+	Múltiples GPUs requeridas

¿Cómo se mide la memoria requerida?

De parámetros a gigabytes de VRAM

Fórmula base: `Memoria = Parámetros × Bytes por parámetro`

FP32 = 4 bytes/param • FP16/BF16 = 2 bytes/param

Ejemplo: Llama 3 8B en FP16 ⟶ 8B × 2 = 16 GB solo pesos

+ KV-cache + activaciones + overhead ≈ +10-20% adicional

Referencia rápida (FP16):

Modelo	VRAM (FP16)	Modelo	VRAM (FP16)
1B	~2 GB	13B	~26 GB
7B	~14 GB	70B	~140 GB (multi-GPU)

⚡ Esta es la razón por la que la cuantización es tan importante.

Cuantización: comprimiendo modelos

¿Menos bits, menos memoria, menos calidad?

La cuantización reduce la precisión numérica de los pesos: 16 bits → 8 bits → 4 bits.

Precisión	Llama 3 8B	Formato común
FP16 (16 bits)	~16 GB	safetensors, HuggingFace
INT8 (8 bits)	~8 GB	BitsAndBytes, GPTQ
Q4_K_M (4 bits)	~4.5 GB	GGUF (Ollama/llama.cpp)
Q2_K (2 bits)	~2.5 GB	GGUF (calidad limitada)

⭐ Q4_K_M y Q5_K_M ofrecen el mejor balance tamaño/calidad. Debajo de 4 bits, la degradación es notable.

Guía práctica de cuantización

Elegir el nivel correcto según tu hardware y tarea

Regla de oro: usa la mayor precisión que tu hardware permita.

Impacto en calidad por tarea

Chat / generación de texto:
Q4_K_M ≈ indistinguible de FP16

Razonamiento matemático:
Degradación notable < Q5

Coding:
Q4_K_M funciona bien; Q3 errores

Extracción estructurada (JSON):
Q4_K_M es suficiente

Recomendación del bootcamp

Desarrollo local:
Q4_K_M como default

Evaluaciones de calidad:
FP16 para benchmarks

Producción:
Depende del volumen y la tolerancia a errores

RTX 3090 (24 GB):
7B en FP16 o 14B en Q4_K_M

La decisión real: Cloud API vs Modelo Local

Esta no es una decisión ideológica sino de ingeniería. Cada opción tiene trade-offs claros.

☁️ A favor de Cloud API

Inicio inmediato sin infraestructura

Calidad state-of-the-art en modelos frontier

Escalamiento automático

No requiere expertise en GPUs/DevOps

Ideal para prototipos y volumen bajo-medio

🖥️ A favor de OSS Local

Privacidad total de datos

Costos predecibles y decrecientes a volumen

Sin dependencia de vendor

Fine-tuning y personalización

Latencia controlable, funciona offline

Matriz de decisión

Tu checklist antes de elegir

Elige Cloud API cuando:

Prototipo/MVP • <10K req/día • Necesitas modelo frontier • Sin equipo DevOps • Presupuesto flexible

Elige OSS cuando:

Datos no pueden salir • >50K req/día • Necesitas fine-tuning • Latencia garantizada • Presupuesto fijo

Zona gris (considera ambos):

10K-50K req/día • Capacidades específicas • Redundancia OSS como fallback • Equipo con algo de infra

⭐ La estrategia más robusta: híbrida. Cloud para lo complejo, OSS para lo repetitivo y sensible.

¿Qué tan lejos están los modelos OSS?

Propietarios vs Open Weight en 2025

La brecha se ha cerrado significativamente pero no ha desaparecido.

Chat, resumen, extracción	Qwen 3.5 4B o 9B ≈ GPT-4o
Coding	Qwen 2.5 Coder, DeepSeek Coder V2 compiten directamente
Razonamiento complejo	Claude Opus, GPT-4o aún llevan ventaja
Razonamiento OSS	DeepSeek R1, Qwen 3.5 27B demuestran que es viable

💡 No necesitas GPT-4o para clasificar tickets. Un 7B fine-tuneado puede hacerlo al 95% de calidad.

Análisis de costo real

Cloud API vs GPU rentada vs GPU propia • 100K req/día

Opción	Costo/mes	Ventaja	Desventaja
Cloud API (GPT-4o)	~$12,000 USD	Cero infra	Escala linealmente
GPU rentada (A100)	~$1,440 USD	8x más barato	Mantener servicio
GPU propia (3090)	~$50 USD	Costo marginal ≈0	Capacidad limitada

Escenario: 100K req/día, ~500 tokens entrada, ~200 tokens salida

⭐ Punto de cruce Cloud↔OSS: entre 5,000 y 20,000 req/día según modelo y tarea.

HuggingFace: el hub del ecosistema

Tu punto de partida para modelos OSS

Qué encontrarás

Model cards con docs y benchmarks

Múltiples formatos por modelo:
safetensors, GGUF, GPTQ, AWQ

Datasets para fine-tuning

Spaces para demos interactivas

Librería Transformers: cargar y
ejecutar modelos con pocas líneas

Cómo navegar eficientemente

Filtrar por tarea:
text-generation, text2text-generation

Ordenar por descargas o trending

Revisar licencia siempre:
Apache 2.0 = libre
Llama License = restricciones

Publishers confiables (cuantizaciones):
unsloth, bartowski, mradermacher

Leyendo un Model Card como AI Engineer

Los campos que importan al evaluar un modelo

Arquitectura y tamaño	Núm. de parámetros, tipo (decoder-only, etc.), contexto máximo ⟶ define requisitos de HW
Licencia	Apache 2.0 (libre), Llama License (>700M usuarios necesitan acuerdo), Gemma License
Benchmarks	MMLU (conocimiento), HumanEval (código), GSM8K (matemáticas), IFEval (instrucciones)
Formatos disponibles	safetensors (GPU), GGUF (llama.cpp/Ollama), GPTQ/AWQ (cuantizado GPU)
Comunidad	Discusiones, issues, contribuciones. Modelo con comunidad activa = mejor soporte

Ollama: inferencia local simplificada

El "Docker" de los modelos de lenguaje

Cómo funciona

Usa llama.cpp como backend

Modelos en formato GGUF cuantizado

Ejecución en CPU+RAM, con
aceleración por GPU si disponible

API compatible con OpenAI
→ Cambiar Cloud↔Local = cambiar URL

Ventajas principales

Instalación trivial (un binario)

Catálogo de modelos preconfigurados

API OpenAI-compatible out-of-the-box

Modelfiles personalizados
(como Dockerfile para modelos)

Gestión automática de memoria

Ollama: cuándo usarlo y cuándo no

✅ Usa Ollama cuando

Desarrollo y necesites LLM local

Probar diferentes modelos rápido

Prototipo que migrará a cloud

Modelo para un solo usuario/equipo

Pipeline sin enviar datos a internet

❌ No uses Ollama cuando

Muchos usuarios concurrentes
(sin batching optimizado)

Máximo throughput en producción
(vLLM es significativamente más rápido)

Control fino de scheduling de requests

Deploy en cluster multi-GPU

Modelos recomendados: qwen3.5:9b • qwen2.5-coder:7b (código) • glm-ocr • mistral:7b

vLLM: serving de alto rendimiento

El estándar de la industria para modelos en producción

Innovación clave: PagedAttention

En lugar de reservar bloques contiguos de memoria para el KV-cache, pagina la memoria como un SO maneja la RAM. Elimina fragmentación y permite 2-4x más requests concurrentes.

Otras optimizaciones:

- Continuous batching (no espera batch completo)
- Tensor parallelism (distribuir entre GPUs)
- Soporte GPTQ/AWQ/FP8
- API compatible con OpenAI out-of-the-box

⭐ Llama 3.1 8B en A100 → >100 req/s con latencias <500ms.

vLLM: cuándo usarlo y cuándo no

El motor de producción

✅ Usa vLLM cuando

Deploy en producción con múltiples usuarios

Maximizar throughput, minimizar latencia

GPUs NVIDIA (A100, H100, RTX 30/40)

Modelos grandes distribuidos multi-GPU

Necesitas API drop-in compatible OpenAI

❌ No uses vLLM cuando

Solo desarrollo local (Ollama es más simple)

Hardware solo CPU sin GPU

Necesitas correr modelos GGUF
(vLLM usa safetensors/GPTQ/AWQ)

Un notebook de Jupyter para experimentar

Configs típicas: A100 80GB → 70B Q4 • 2xA100 → 70B FP16 • RTX 3090 → 14B Q4

SGLang: frontend + backend optimizado

La frontera de rendimiento en inferencia

Innovación clave: RadixAttention

Cachea y reutiliza prefijos de prompts entre requests. Si 100 usuarios comparten el mismo system prompt, SGLang lo procesa UNA vez y reutiliza el KV-cache.

Speedup 2x-5x en workloads con prompts repetitivos:

- Agentes con tools (mismo system prompt)
- RAG con contexto compartido
- Chatbots con system prompt fijo

Frontend DSL:

DSL en Python para generación estructurada: branching, selección entre opciones, JSON schema constraints.

SGLang: cuándo es la mejor opción

✅ Usa SGLang cuando

Prompts con prefijos repetitivos (system prompts largos, RAG compartido)

Generación estructurada de alto perf. (JSON, function calling)

Máximo throughput en agentes con herramientas y contexto compartido

Evaluando modelos con miles de requests similares (benchmarks)

❌ No uses SGLang cuando

Prompts todos únicos sin prefijos compartidos (pierdes RadixAttention)

Necesitas el ecosistema más maduro de vLLM

Tu equipo no tiene familiaridad con el DSL de SGLang

La simplicidad de Ollama es suficiente

⭐ SGLang y vLLM convergen en features. La diferencia clave: RadixAttention + frontend estructurado.

¿Cuál herramienta para cuál escenario?

Ollama vs vLLM vs SGLang

Criterio	Ollama	vLLM	SGLang
Setup	⭐ Un binario	pip install + config	pip install + config
Throughput	Bajo (sin batching)	Alto (PagedAttention)	Muy alto (RadixAttn)
Formato modelos	GGUF	safetensors/GPTQ/AWQ	safetensors/GPTQ/AWQ
API OpenAI	✅	✅	✅
Multi-GPU	Limitado	⭐ Excelente	⭐ Excelente
Ideal para	Desarrollo	Producción general	Prompts repetitivos

⭐ Recomendación: Ollama para desarrollo, evalúa vLLM o SGLang para producción según tu workload.

Extra: llama.cpp

La base de todo el ecosistema local

Proyecto fundacional de inferencia local de LLMs, escrito en C/C++ por Georgi Gerganov. Es el motor que corre detrás de Ollama y muchas otras herramientas.

¿Por qué importa?	Primer proyecto en hacer viable inferencia de LLMs en hardware de consumidor. Introdujo el formato GGUF.
Soporte de HW	CPU puro, NVIDIA (CUDA), AMD (ROCm), Apple Silicon (Metal), NPUs
¿Cuándo usarlo directo?	Máximo control, hardware no-NVIDIA, embeber LLM en app C/C++, mínima dependencia
En este bootcamp	No lo usamos directamente — Ollama lo abstrae. Pero es importante saber que existe como capa más baja.

Ollama: setup en 3 comandos

```
# 1. Instalar Ollama (Linux/Mac/Windows)
curl -fsSL https://ollama.ai/install.sh | sh

# 2. Descargar un modelo
ollama pull llama3.1:8b

# 3. Verificar que funciona
ollama run llama3.1:8b "Hola, funciona?"

# Ver modelos descargados
ollama list

# Otros modelos utiles:
ollama pull qwen2.5-coder:7b    # Para codigo
ollama pull qwem3.5:9b            # Alternativa general
ollama pull nomic-embed-text     # Embeddings locales
```

⭐ Ollama corre como servicio en http://localhost:11434. API REST compatible con OpenAI en /v1/.

Ollama: consumiendo la API desde Python

```python
from openai import OpenAI

# Ollama expone API compatible con OpenAI
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama",  # No requiere key real
)

response = client.chat.completions.create(
    model="llama3.1:8b",
    messages=[
        {"role": "system", "content": "Responde en espanol"},
        {"role": "user", "content": "Que es RAG?"},
    ],
    temperature=0.3,
)
print(response.choices[0].message.content)
```

Ollama: Modelfile personalizado

Como un Dockerfile pero para modelos

```
# Archivo: Modelfile.docops
FROM llama3.1:8b

SYSTEM """Eres DocOps, un asistente de documentacion
empresarial. Respondes en espanol. Citas la
fuente del documento cuando la tienes."""

PARAMETER temperature 0.3
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
```

```
# Crear y usar el modelo personalizado
ollama create docops-agent -f Modelfile.docops
ollama run docops-agent "Como organizo mis docs?"
```

⭐ Preconfigura system prompt y parámetros del DocOps Agent sin hardcodearlos en Python.

Ollama: embeddings sin API externa

```python
from openai import OpenAI

client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

# Generar embeddings locales
response = client.embeddings.create(
    model="nomic-embed-text",
    input=[
        "Politica de devoluciones de la empresa",
        "Proceso de onboarding para empleados",
        "Configuracion del servidor de produccion",
    ]
)

for i, emb in enumerate(response.data):
    print(f"Doc {i}: {len(emb.embedding)} dims")
```

🔒 Con esto tu pipeline RAG completo puede correr sin ninguna API externa.

vLLM: levantando un servidor de producción

```
# Instalar vLLM
pip install vllm

# Lanzar servidor con modelo de HuggingFace
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --host 0.0.0.0 \
    --port 8000 \
    --max-model-len 8192 \
    --gpu-memory-utilization 0.9

# Con cuantizacion AWQ (menos VRAM):
vllm serve TechxGenus/Llama-3.1-8B-Instruct-AWQ \
    --quantization awq

# Multi-GPU (70B en 2 GPUs):
vllm serve meta-llama/Llama-3.1-70B-Instruct \
    --tensor-parallel-size 2
```

⭐ vLLM descarga automáticamente el modelo de HuggingFace. Servidor en http://localhost:8000/v1

vLLM: mismo código, mayor throughput

```python
from openai import OpenAI

# Conectar al servidor vLLM
client = OpenAI(base_url="http://localhost:8000/v1", api_key="x")

# Exactamente la misma API que Ollama y OpenAI
response = client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "system", "content": "Eres DocOps."},
        {"role": "user", "content": "Extrae los puntos clave"},
    ],
    temperature=0.2,
)

# JSON mode (outputs estructurados)
response = client.chat.completions.create(
    ..., response_format={"type": "json_object"})
```

⭐ El campo model debe coincidir exactamente con el nombre usado al lanzar el servidor.

vLLM: configuración de producción

```
# Configuracion de produccion optimizada
vllm serve meta-llama/Llama-3.1-8B-Instruct \
    --host 0.0.0.0 --port 8000 \
    --max-model-len 8192 \
    --gpu-memory-utilization 0.92 \
    --max-num-seqs 64 \
    --enable-prefix-caching \
    --dtype bfloat16
```

```
# Flags importantes:
# --max-num-seqs       requests concurrentes en batch
# --enable-prefix-caching cachea system prompts
# --gpu-memory-utilization % de VRAM a usar (0.9-0.95)
# --dtype bfloat16     mejor precision que float16
```

⭐ --enable-prefix-caching es clave para DocOps: mismo system prompt procesado una sola vez.

SGLang: setup del servidor

RadixAttention se activa automáticamente

```
# Instalar SGLang
pip install "sglang[all]"

# Lanzar servidor (API OpenAI-compatible)
python -m sglang.launch_server \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --host 0.0.0.0 --port 30000 \
    --mem-fraction-static 0.9

# Con cuantizacion:
python -m sglang.launch_server \
    --model ... --quantization awq

# Multi-GPU:
python -m sglang.launch_server \
    --model ... --tp 2
```

⭐ Servidor en http://localhost:30000/v1. RadixAttention cachea prefijos sin config adicional.

SGLang: consumo estándar

Misma interfaz que vLLM y Ollama

```python
from openai import OpenAI

client = OpenAI(base_url="http://localhost:30000/v1", api_key="x")

response = client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "system", "content": "Eres DocOps."},
        {"role": "user", "content": "Clasifica este ticket"},
    ],
    temperature=0.1,
)

# Por que SGLang brilla aqui?
# 1000 requests con MISMO system prompt:
# RadixAttention: 1 procesamiento + 999 cache hits
# vLLM (sin prefix caching): ~1000 procesamientos
```

SGLang: DSL para generación estructurada

sgl.select fuerza selección entre opciones, sin parsing de texto

```python
import sglang as sgl

@sgl.function
def clasificar_ticket(s, ticket_texto):
    s += sgl.system("Eres un clasificador.")
    s += sgl.user(f"Clasifica: {ticket_texto}")
    s += sgl.assistant(sgl.select(
        "categoria",
        ["Bug", "Feature", "Soporte", "Pregunta"]))

state = clasificar_ticket.run(
    ticket_texto="La app se cierra al subir fotos",
    backend=sgl.RuntimeEndpoint("http://localhost:30000"))
print(state["categoria"])  # "Bug"
```

⭐ Mucho más robusto que esperar que el modelo genere exactamente el formato correcto.

Benchmark: Ollama vs vLLM vs SGLang

50 requests idénticas, mismo modelo

```python
import time
from openai import OpenAI

def benchmark(base_url, model, n=50):
    client = OpenAI(base_url=base_url, api_key="x")
    msgs = [{"role":"user","content":"Que es Docker?"}]
    start = time.time()
    for _ in range(n):
        client.chat.completions.create(model=model, messages=msgs)
    elapsed = time.time() - start
    print(f"{n/elapsed:.1f} req/s")
```

Resultado típico RTX 3090 (secuencial, 1 a 1):

Ollama: ~0.4 req/s **vLLM:** ~1.4 req/s **SGLang:** ~2.3 req/s ← prefix caching

DocOps Agent: módulo de inferencia local

Estructura actualizada del proyecto

```
docops-agent/
├── core/
│   ├── llm_client.py          # Clase 1
│   └── tokenlab.py            # Clase 2
├── prompting/
│   └── promptkit.py           # Clase 3
├── contracts/
│   └── structured_output.py   # Clase 4
├── rag/
│   ├── embeddings.py ... cache.py # Clases 5-7
├── inference/                 ← NUEVO
│   └── local_adapter.py       # Clase 8
└── .env
```

⭐ Hoy construimos inference/local_adapter.py: intercambiar Cloud ↔ Local sin cambiar el resto.

local_adapter.py: interfaz unificada

inference/local_adapter.py

inference/local_adapter.py

```python
from openai import OpenAI
from enum import Enum

class InferenceBackend(Enum):
    CLOUD  = "cloud"
    OLLAMA = "ollama"
    VLLM   = "vllm"
    SGLANG = "sglang"

BACKEND_URLS = {
    InferenceBackend.OLLAMA: "http://localhost:11434/v1",
    InferenceBackend.VLLM:   "http://localhost:8000/v1",
    InferenceBackend.SGLANG: "http://localhost:30000/v1",
}

def get_client(backend, api_key="not-needed"):
    if backend == InferenceBackend.CLOUD:
        return OpenAI()  # usa OPENAI_API_KEY del .env
    return OpenAI(base_url=BACKEND_URLS[backend], api_key=api_key)
```

Integrando con el proyecto existente

El resto del proyecto (RAG, prompts, contracts) no cambia

```python
from inference.local_adapter import get_client, InferenceBackend

# Desarrollo local con Ollama
client = get_client(InferenceBackend.OLLAMA)
response = client.chat.completions.create(
    model="llama3.1:8b",
    messages=[{"role":"user","content":"Resume esto"}],
)

# Cambiar a produccion con vLLM:
# client = get_client(InferenceBackend.VLLM)
# model = "meta-llama/Llama-3.1-8B-Instruct"

# Cambiar a cloud:
# client = get_client(InferenceBackend.CLOUD)
# model = "gpt-4o-mini"
```

🔑 El truco: Ollama, vLLM y SGLang exponen APIs OpenAI-compatible. Solo cambia la URL.

Resumen + Próxima Clase

Takeaways de la Clase 8

Modelos OSS: privacidad, control, costos predecibles. Trade-off: tú gestionas la infra.

Memoria GPU:
Parámetros × Bytes. Q4_K_M reduce a ~25%.

Cloud vs OSS: decisión de ingeniería.
Usa la matriz según volumen y privacidad.

Herramientas:
Ollama → dev | vLLM → prod | SGLang → prod opt.
Las 3 exponen API OpenAI-compatible.

Tarea: Clase 8

1. Instalar Ollama y descargar qwen3.5:0.8b
2. Crear Modelfile para DocOps Agent
3. Implementar local_adapter.py
4. Correr pipeline RAG (Clase 7) usando Ollama en vez de OpenAI
5. Medir latencia: local vs cloud

Clase 9: De workflows a agentes → Pipelines lineales a sistemas que razonan, deciden y actúan